



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Grafos
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero – “B”
Alumnos participantes:	Ortiz Cunalata Alina Elizabeth
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en la programación con grafos.

Específicos:

- Identificar los conceptos fundamentales de los grafos, incluyendo vértices, aristas y tipos de grafos, para su correcta aplicación en programación.
- Implementar estructuras de datos adecuadas para representar grafos mediante listas de adyacencia y matrices de adyacencia.
- Aplicar algoritmos de recorrido y búsqueda en grafos, como búsqueda en profundidad (DFS) y búsqueda en amplitud (BFS), para resolver problemas computacionales.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC Computadora y acceso al internet

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Implementar el caso de estudio del Mapa del campus y búsqueda de rutas.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de la vida real, en este caso calcular las rutas entre dos ubicaciones., utilizando grafos.



Introducción

Un grafo es una estructura de datos no lineal que consiste en un conjunto de vértices (nodos) conectados por aristas (edges). Para este caso, se modeló el Campus Huachi de la UTA como un grafo no dirigido y ponderado, donde los nodos representan edificios o áreas del campus, las aristas representan caminos peatonales, y los pesos representan distancias en metros.

Métodos Implementados

1. agregarNodo() - Agrega un vértice al grafo

```
public void agregarNodo(String id, String nombre) {  
    Nodo nodo = new Nodo(id, nombre);  
    nodos.put(id, nodo);  
    adyacencia.put(id, new ArrayList<>());  
}
```

2. agregarArista() - Agrega una conexión bidireccional con peso

```
public void agregarArista(String origen, String destino, int peso) {  
  
    adyacencia.get(origen).add(new Arista(destino, peso));  
    adyacencia.get(destino).add(new Arista(origen, peso));  
}
```

3. bfs() - Búsqueda en anchura (encuentra ruta con menos paradas)

```
public List<String> bfs(String inicio, String fin) {  
    Queue<List<String>> cola = new LinkedList<>();  
    Set<String> visitados = new HashSet<>();  
    List<String> caminoInicial = new ArrayList<>();  
    caminoInicial.add(inicio);  
    cola.add(caminoInicial);  
    visitados.add(inicio);  
    while (!cola.isEmpty()) {  
        List<String> camino = cola.poll();  
        String actual = camino.get(camino.size() - 1);  
        if (actual.equals(fin)) {  
            return camino;  
        }  
        for (Arista arista : adyacencia.get(actual)) {  
            if (!visitados.contains(arista.destino)) {  
  
                visitados.add(arista.destino);  
                List<String> nuevoCamino = new ArrayList<>(camino);  
                nuevoCamino.add(arista.destino);  
                cola.add(nuevoCamino);  
            }  
        }  
    }  
    return null;  
}
```



3. dijkstra() - Camino mínimo (encuentra ruta con menor distancia)

```
public List<String> dijkstra(String inicio, String fin) {
    Map<String, Integer> distancias = new HashMap<>();
    Map<String, String> anteriores = new HashMap<>();
    PriorityQueue<String> cola =
        new PriorityQueue<>(Comparator.comparingInt(distancias::get));
    for (String nodo : nodos.keySet()) {
        distancias.put(nodo, Integer.MAX_VALUE);
    }
    distancias.put(inicio, value: 0);
    cola.add(inicio);
    while (!cola.isEmpty()) {
        String actual = cola.poll();
        for (Arista arista : adyacencia.get(actual)) {
            int nuevaDistancia =
                distancias.get(actual) + arista.peso;
            if (nuevaDistancia < distancias.get(arista.destino)) {
                distancias.put(arista.destino, nuevaDistancia);
                anteriores.put(arista.destino, actual);
                cola.add(arista.destino);
            }
        }
    }
    List<String> camino = new ArrayList<>();
    String actual = fin;
    while (actual != null) {
        camino.add(index: 0, actual);
        actual = anteriores.get(actual);
    }
    return camino;
}
```

4. Prueba de Ruta: Universidad → Facultad de Alimentos

```
PS C:\Users\ALINA\Downloads\Ape4-Grafos-Completo> cd 'c:\Users\ALINA\Downloads\Ape4-Grafos-Completo'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.11-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ALINA\AppData\Roaming\Cod e\workspaceStorage\22dac0c98674df192e2391a9ccc29f6b\redhat.java\jdt_ws\Ape4-Grafos-Completo_527c014e\bin' 'Ape4_Grafos'
===== BFS =====
Universidad (uta) -> Comedor (comedor) -> Estadio (estadio) -> Facultad de Alimentos (alimentos)

===== DIJKSTRA =====
Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Facultad de Alimentos (alimentos)
PS C:\Users\ALINA\Downloads\Ape4-Grafos-Completo>
```

Resultados obtenidos:

Algoritmo	Camino	Distancia	Tiempo	Paradas
BFS	Universidad → Alimentos	45 m	0.56 min	0
Dijkstra	Universidad → Alimentos	45 m	0.56 min	0

Comparación BFS vs Dijkstra

Característica	BFS	Dijkstra
Estructura	Cola (FIFO)	Cola de prioridad
Considera pesos	No	Sí
Objetivo	Menos paradas	Menor distancia
Complejidad	$O(V + E)$	$O((V + E) \log V)$

2.8 Habilidades blandas empleadas en la práctica



- ☒ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Identificar los conceptos fundamentales de los grafos, incluyendo vértices, aristas y tipos de grafos, para su correcta aplicación en programación.
- Implementar estructuras de datos adecuadas para representar grafos mediante listas de adyacencia y matrices de adyacencia.
- Aplicar algoritmos de recorrido y búsqueda en grafos, como búsqueda en profundidad (DFS) y búsqueda en amplitud (BFS), para resolver problemas computacionales.

2.10 Recomendaciones

- Practicar constantemente la implementación de grafos utilizando diferentes estructuras de representación para mejorar la comprensión del tema.
- Profundizar en el estudio de algoritmos avanzados de grafos, como Dijkstra, Floyd-Warshall y Kruskal, para ampliar las capacidades de resolución de problemas.
- Desarrollar proyectos prácticos relacionados con redes, rutas de transporte o sistemas de recomendación que permitan aplicar los conocimientos adquiridos sobre grafos.

2.11 Referencias bibliográficas

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

2.12 Anexos

<https://github.com/Alí-20-l/Guia-Ape4.git>